

Databases

Scalaire functies

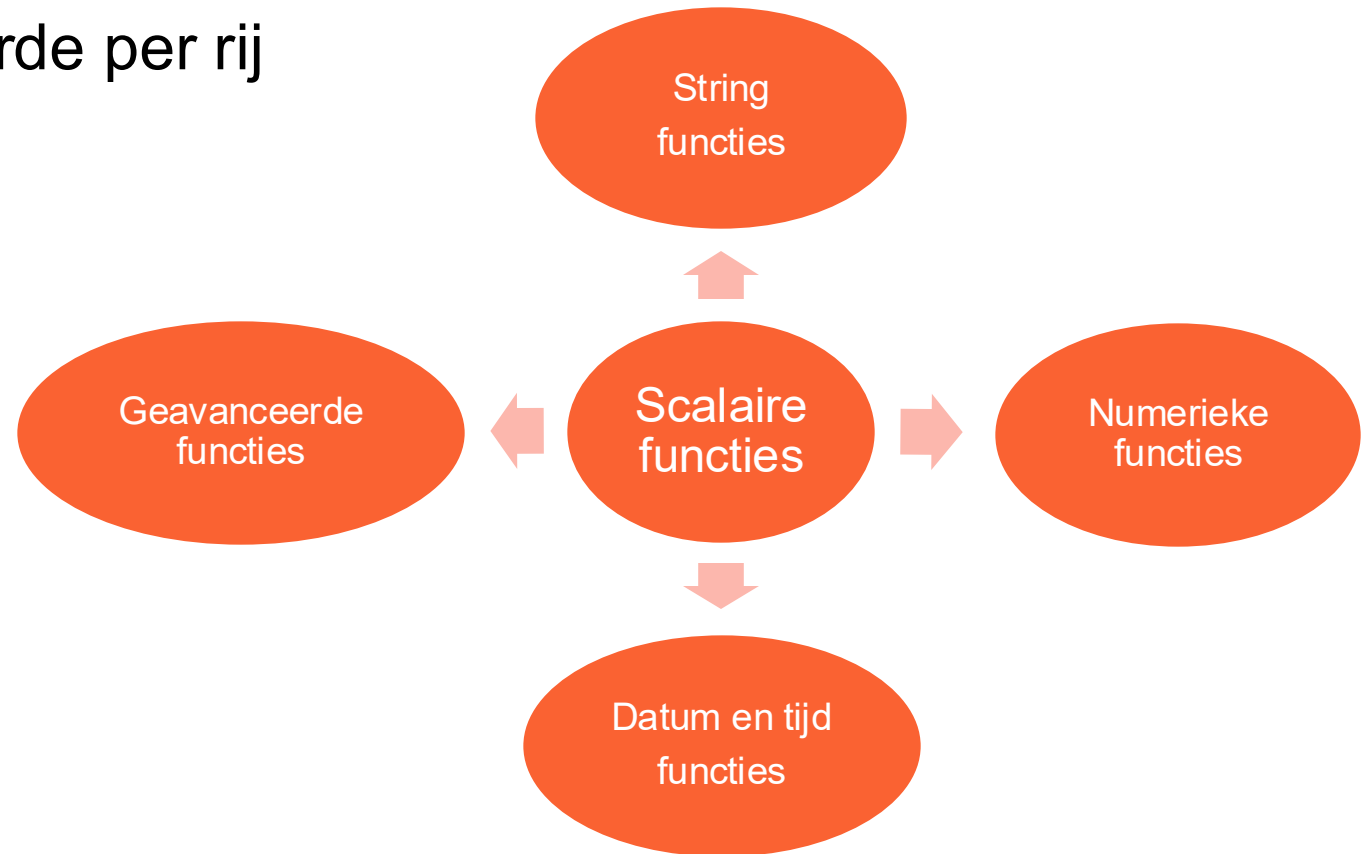
Inhoud

- Scalaire functies
 - Stringfuncties
 - Numerieke / wiskundige functies
 - Datum- en tijd functies
 - Geavanceerde functies
- ORDER BY component (deel 2)



Wat is een scalaire functie ?

- Hebben 0, 1 of meer parameters
- Resultaat: 1 uitvoerwaarde per rij
- Werken slechts op 1 rij



Stringfuncties

We bespreken de meest gebruikte:

CONCAT met +
LTRIM()
SPACE()
LOWER()
LEFT()
LEN()
REPLACE()

- CONCAT_WS()
- TRIM()
- SUBSTRING()

CONCAT()
RTRIM()

UPPER()
RIGHT()
CHARINDEX()

Meer informatie over de andere functies:

https://www.w3schools.com/sql/sql_ref_sqlserver.asp

Stringfuncties: om strings samen te voegen

Gebruik:

- Om één of meerdere stringvelden samen te voegen tot één **tijdelijk** veld.

Te gebruiken stringfuncties:

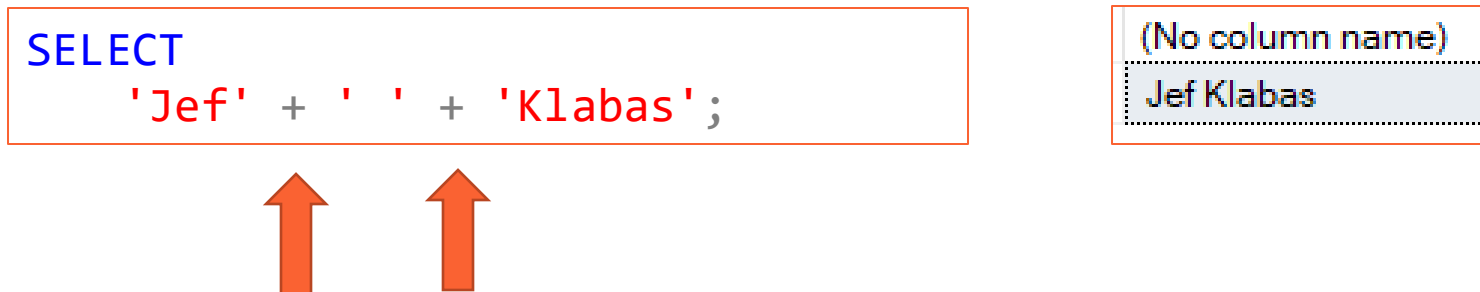
- CONCAT met +
- CONCAT()
- CONCAT_WS()

Stringfuncties: om strings samen te voegen

Manier 1: CONCAT met +

Syntax:

- `string1 + string2 + string_n`
- Betekenis:
één of meerdere strings samenvoegen tot één **tijdelijk** veld via het concatenatie-teken '+'.
- Voorbeeld:



Stringfuncties: om strings samen te voegen

Manier 1: CONCAT met +

Voorbeeld:

```
SELECT  
    voornaam + ' ' + naam AS 'speler'  
FROM Tennis.Speler;
```

voornaam	naam
Rudi	Elfring
Raf	Permentier
Geert-Jan	Wijers
Bertha	Niewenburg
Marjan	Cools
Carla	Cools
Danny	Bischoff
Eric	Bakker, de
Marc	Bohemen, van
Pieter	Hofland
Peter	Meuleman
Patrick	Permentier
Daniëlla	Moerman
Ingrid	Baalen, van
Lydia	Van_Den_Berg



speler	
Rudi	Elfring
Raf	Permentier
Geert-Jan	Wijers
Bertha	Niewenburg
Marjan	Cools
Carla	Cools
Danny	Bischoff
Eric	Bakker, de
Marc	Bohemen, van
Pieter	Hofland
Peter	Meuleman
Patrick	Permentier
Daniëlla	Moerman
Ingrid	Baalen, van
Lydia	Van_Den_Berg

100% is het nog niet,
nog even geduld

Stringfuncties: om strings samen te voegen

Manier 2: CONCAT()

Syntax:

- `CONCAT(string1, string2, ..., string_n)`

Betekenis:

één of meerdere strings samenvoegen tot één **tijdelijk** veld via de functie `CONCAT()`.

Voorbeeld:

```
SELECT  
    CONCAT('Jef', ' ', 'Klabas');
```

(No column name)

Jef Klabas

```
SELECT  
    CONCAT('Jef', 'Klabas');
```

(No column name)

JefKlabas

Stringfuncties: om strings samen te voegen

Manier 2: CONCAT()

Voorbeeld:

```
SELECT  
    CONCAT(voornaam, ' ', naam) AS 'speler'  
FROM Tennis.Speler;
```

voornaam	naam
Rudi	Elfring
Raf	Permentier
Geert-Jan	Wijers
Bertha	Niewenburg
Marjan	Cools
Carla	Cools
Danny	Bischoff
Eric	Bakker, de
Marc	Bohemen, van
Pieter	Hofland
Peter	Meuleman
Patrick	Permentier
Daniëlla	Moerman
Ingrid	Baalen, van
Lydia	Van_Den_Berg



speler	
Rudi	Elfring
Raf	Permentier
Geert-Jan	Wijers
Bertha	Niewenburg
Marjan	Cools
Carla	Cools
Danny	Bischoff
Eric	Bakker, de
Marc	Bohemen, van
Pieter	Hofland
Peter	Meuleman
Patrick	Permentier
Daniëlla	Moerman
Ingrid	Baalen, van
Lydia	Van_Den_Berg

100% is het nog niet,
nog even geduld

Stringfuncties: om strings samen te voegen

Manier 3: CONCAT_WS()

Gebruik:

- Voegt twee of meer strings samen tot één **tijdelijk** veld met een scheidingsteken toe.

Syntax:

- CONCAT_WS(scheidingsteken, string1, string2, ..., string_n)

Voorbeeld:

```
SELECT  
  CONCAT_WS('-', 'Jef', 'Klabas', 'Pipo', 'Clown');
```



(No column name)

Jef-Klabas-Pipo-Clown



```
SELECT  
  CONCAT_WS('*', 'Jef', 'Klabas', 'Pipo', 'Clown');
```



(No column name)

Jef*Klabas*Pipo*Clown



Stringfuncties: om strings samen te voegen

Manier 3: CONCAT_WS()

Voorbeeld:

```
SELECT
    CONCAT_WS('-', voornaam, naam) AS 'speler'
FROM Tennis.Speler;
```

voornaam	naam
Rudi	Elfring
Raf	Permentier
Geert-Jan	Wijers
Bertha	Niewenburg
Marjan	Cools
Carla	Cools
Danny	Bischoff
Eric	Bakker, de
Marc	Bohemen, van
Pieter	Hofland
Peter	Meuleman
Patrick	Permentier
Daniëlla	Moerman
Ingrid	Baalen, van
Lydia	Van_Den_Berg



speler	
Rudi	-Elfring
Raf	-Permentier
Geert-Jan	-Wijers
Bertha	-Niewenburg
Marjan	-Cools
Carla	-Cools
Danny	-Bischoff
Eric	-Bakker, de
Marc	-Bohemen, van
Pieter	-Hofland
Peter	-Meuleman
Patrick	-Permentier
Daniëlla	-Moerman
Ingrid	-Baalen, van
Lydia	-Van_Den_Berg

100% is het nog niet,
nog even geduld

Stringfuncties: om strings samen te voegen

Welke functies best gebruiken?

- CONCAT()
- CONCAT_WS()
- Zorgen zelf dat bepaalde problemen opgelost worden (meer uitleg later)
- Indien je + gebruikt om attributen te samen te voegen, moet je bepaalde problemen zelf oplossen (meer uitleg later)

Stringfuncties: om spaties uit strings tijdelijk te verwijderen

Gebruik:

- Om leidende-, volgs spaties en eventuele andere tekens te verwijderen.

Te gebruiken stringfuncties:

- LTRIM(): Verwijder **de leidende spaties** uit een string.
- RTRIM(): Verwijder **volgs spaties** uit een string.
- TRIM(): **Leidende** en **volgs spaties** (standaard) of **andere opgegeven tekens** uit een string verwijderen.

Stringfuncties: om spaties uit strings tijdelijk te verwijderen

Syntax:

- LTRIM(string)
- RTRIM(string)
- TRIM([karakter FROM]string)

Voorbeeld:

```
SELECT
    '*' + naam + '*',
    '*' + LTRIM(naam) + '*' AS 'alleenLinks',
    '*' + RTRIM(naam) + '*' AS 'alleenRechts',
    '*' + LTRIM(RTRIM(naam)) + '*' AS 'beiden',
    '*' + TRIM(naam) + '*' AS 'beiden via trim()'
FROM Tennis.Speler;
```

Stringfuncties: om spaties uit strings tijdelijk te verwijderen

Voorbeelden:

(No column name)	alleenLinks	alleenRechts	beiden	beiden via trim()
*Elfring *	*Elfring *	*Elfring*	*Elfring*	*Elfring*
*Permentier *	*Permentier *	*Permentier*	*Permentier*	*Permentier*
*Wijers *	*Wijers *	*Wijers*	*Wijers*	*Wijers*
*Niewenburg *	*Niewenburg *	*Niewenburg*	*Niewenburg*	*Niewenburg*
* Cools *	*Cools *	* Cools*	*Cools*	*Cools*
* Cools *	*Cools *	* Cools*	*Cools*	*Cools*
*Bischoff *	*Bischoff *	*Bischoff*	*Bischoff*	*Bischoff*
*Bakker, de *	*Bakker, de *	*Bakker, de*	*Bakker, de*	*Bakker, de*
*Bohemen, van *	*Bohemen, van *	*Bohemen, van*	*Bohemen, van*	*Bohemen, van*
*Hofland *	*Hofland *	*Hofland*	*Hofland*	*Hofland*
*Meuleman *	*Meuleman *	*Meuleman*	*Meuleman*	*Meuleman*
*Permentier *	*Permentier *	*Permentier*	*Permentier*	*Permentier*
*Moerman *	*Moerman *	*Moerman*	*Moerman*	*Moerman*
*Baalen, van *	*Baalen, van *	*Baalen, van*	*Baalen, van*	*Baalen, van*
*Van_Den_Berg *	*Van_Den_Berg *	*Van_Den_Berg*	*Van_Den_Berg*	*Van_Den_Berg*

Stringfuncties: om spaties uit strings tijdelijk te verwijderen en strings samen te voegen

Voorbeelden:

```
SELECT  
    LTRIM(RTRIM(voornaam)) + ' ' + LTRIM(RTRIM(naam)) AS  
'speler'  
FROM Tennis.Speler;
```

```
SELECT  
    TRIM(voornaam) + ' ' + TRIM(naam) AS 'speler'  
FROM Tennis.Speler;
```

```
SELECT  
    CONCAT(TRIM(voornaam), ' ', TRIM(naam)) AS 'speler'  
FROM Tennis.Speler;
```

```
SELECT  
    CONCAT_WS(' ', TRIM(voornaam), TRIM(naam)) AS 'speler'  
FROM Tennis.Speler;
```

speler
Rudi Elfring
Raf Permentier
Geert-Jan Wijers
Bertha Niewenburg
Marjan Cools
Carla Cools
Danny Bischoff
Eric Bakker, de
Marc Bohemen, van
Pieter Hofland
Peter Meuleman
Patrick Permentier
Daniëlla Moerman
Ingrid Baalen, van
Lydia Van_Den_Berg

Stringfuncties: TRIM() met een ander karakter

Voorbeeld:

```
SELECT  
    TRIM('rgba ' FROM voornaam)  
FROM Tennis.Speler;
```

Betekenis: de karakters 'r', 'g', 'b', 'a' en spatie die voor of achteraan voorkomen worden tijdelijk verwijderd.

voornaam
Rudi
Raf
Geert-Jan
Bertha
Marjan
Carla
Danny
Eric
Marc
Pieter
Peter
Patrick

(No column name)
udi
f
eert-Jan
erth
Marjan
Carl
Danny
Eric
Marc
Piete
Pete
Patrick

Stringfuncties: SPACE()

Gebruik:

- Om een opgegeven aantal **spaties** tussen strings te plaatsen.

Syntax:

- `SPACE(aantal)`

Voorbeeld:

```
SELECT
    postcode + SPACE(5) + LTRIM(plaats) AS 'woonplaats'
FROM Tennis.Speler;
```

woonplaats

3575NH	Den Haag
1234KK	Den Haag
9758VB	Den Haag
6584WO	Rijswijk
8457DK	Zoetermeer
1294QK	Leiden
9629CD	Den Haag
4444LJ	Rijswijk
4377CB	Den Haag
1812UP	Den Haag
5746OP	Voorburg
6494SG	Den Haag
9437AO	Zoetermeer
6392LK	Rotterdam
1012AB	Amsterdam

Stringfuncties: LOWER() – UPPER()

Gebruik:

- Ongeacht de inhoud van een string alle letters **als kleine of hoofdletters** tonen afhankelijk van de gebruikte functie.

Syntax:

- LOWER(*string*)
- UPPER(*string*)

Betekenis:

- String in *kleine letters* tonen
- String in *hoofdletters* tonen

Stringfuncties: LOWER() – UPPER()

Voorbeelden:

```
SELECT  
    LOWER('Deze KARAKTERS worden IN KLEINE LETTERS getoond');
```

(No column name)

deze karakters worden in kleine letters getoond

```
SELECT  
    naam,  
    LOWER(naam)  
FROM Tennis.Speler;
```

naam	(No column name)
Elfring	elfring
Permentier	permentier
Wijers	wijers
Niewenburg	niewenburg
Cools	cools

```
SELECT  
    naam,  
    UPPER(naam)  
FROM Tennis.Speler;
```

naam	(No column name)
Elfring	ELFRING
Permentier	PERMENTIER
Wijers	WIJERS
Niewenburg	NIEWENBURG
Cools	COOLS

Stringfuncties: om delen van een string te selecteren

Gebruik:

- Een **deel** (= aantal karakters) uit een stringveld selecteren, **beginnend vanaf links of rechts**.
- Of een **deel** uit een stringveld selecteren, **beginnend vanaf de opgegeven startpositie**.

Syntax:

- `LEFT(string, aantal_karakters)`
- `RIGHT(string, aantal_karakters)`
- `SUBSTRING(string, startpositie, aantal_karakters)`

Stringfuncties: om delen van een string te selecteren

Voorbeelden:

```
SELECT
    naam,
    LEFT(naam,3)
FROM Tennis.Speler;
```

naam	(No column name)
Elfring	Elf
Permentier	Per
Wijers	Wij
Niewenburg	Nie
Cools	Coo

```
SELECT
    naam,
    RIGHT(naam, 5)
FROM Tennis.Speler;
```

naam	(No column name)
Elfring	
Permentier	
Wijers	
Niewenburg	
Cools	



```
SELECT
    naam,
    LEFT(naam,3),
    RIGHT(TRIM(naam), 5)
FROM Tennis.Speler;
```

naam	(No column name)	(No column name)
Elfring	Elf	fring
Permentier	Per	ntier
Wijers	Wij	ijers
Niewenburg	Nie	nburg
Cools	Coo	Cools

Stringfuncties: om delen van een string te selecteren

Voorbeeld:

```
SELECT  
    naam,  
    SUBSTRING(naam, 2, 3)  
FROM Tennis.Speler;
```

naam	(No column name)
Elfring	lfr
Permentier	erm
Wijers	ije
Niewenburg	iew
Cools	ool
Cools	ool

- 3 karakters selecteren vanaf positie 2

Stringfuncties: aantal karakters van een string bepalen

Gebruik:

- Geeft de **lengte** van een string terug.

Syntax:

- `LEN(string)`

Voorbeeld:

```
SELECT  
    naam,  
    LEN(naam)  
FROM Tennis.Speler;
```

naam	(No column name)
Elfring	7
Permentier	10
Wijers	6
Niewenburg	10
Cools	5

Stringfuncties: positie van opgegeven tekens in een string

Gebruik:

- Geeft de **positie** van de te zoeken karakters in een string terug.

Syntax:

- CHARINDEX(*te zoeken string*, *string*, [*start*])

Voorbeeld:

```
SELECT  
  CHARINDEX('a', 'Jef Klabas');
```

(No column name)

7

```
SELECT  
  CHARINDEX('a', 'Jef Klabas', 8);
```

(No column name)

9

Stringfunctie: tijdelijk vervangen van karakters

Gebruik:

- vervangt de opgegeven karakters binnen een string met een nieuwe tekenreeks.

Te gebruiken functie :

- REPLACE()

Stringfunctie: tijdelijk vervangen van karakters

Syntax:

- REPLACE(string, oude string, nieuwe string)

Voorbeelden:

```
SELECT  
  REPLACE('SQL Tutorial', 'Tutorial', 'handleiding');
```

(No column name)

SQL handleiding

```
SELECT  
  REPLACE(divisie, divisie, TRIM(divisie) + ' divisie'),  
  divisie  
FROM Tennis.Team;
```

(No column name)

divisie

ere divisie

ere

tweede divisie

tweede

Numerieke/wiskundige functies

We bespreken de meest gebruikte:

ROUND()

RAND()

FLOOR()

CEILING()

Meer informatie over de andere functies:

https://www.w3schools.com/sql/sql_ref_sqlserver.asp

Numerieke / wiskundige functies: afgerond, volgend - of vorig geheel getal bepalen

Gebruik:

- Geeft een getal dat is **afgerond** op een bepaald aantal decimalen.
- Geeft de **grootste** gehele getalwaarde die **kleiner of gelijk** is aan het opgegeven getal.
- Geeft de **kleinste** gehele getalwaarde die **groter of gelijk** is aan het opgegeven getal

Te gebruiken functies:

- ROUND()
- FLOOR()
- CEILING()

Numerieke / wiskundige functies: afgerond, volgend - of vorig geheel getal bepalen

Syntax:

- `ROUND(getal, aantalDecimalen, [operatie])`
- `FLOOR(getal)`
- `CEILING(getal)`

Voorbeeld:

```
SELECT  
    ROUND(148.85,1) AS 'afgerond',  
    FLOOR(148.85) AS 'ondergrens',  
    CEILING(148.85) AS 'bovengrens';
```

afgerond	ondergrens	bovengrens
148.90	148	149

Numerieke / wiskundige functies: afgerond getal bepalen

Syntax:

- `ROUND(getal, aantalDecimalen, [operatie])`

Uitleg [*operatie*]:

- kan 0 [standaard] of een andere numerieke waarde hebben
- `= 0`: het resultaat wordt afgerond naar het aantal opgegeven decimalen
- `<> 0`: het resultaat wordt afgekapt naar het aantal opgegeven decimalen

Numerieke / wiskundige functies: afgerond getal bepalen

Voorbeelden:

- Afronden op 2 cijfers na de komma

```
SELECT ROUND(1245.5556,2);
```

(No column name)

1245.5600

- Afkappen op 2 cijfers na de komma

```
SELECT ROUND(1245.5556,2,1);
```

(No column name)

1245.5500

- Afronden op 1 cijfer na de komma

```
SELECT ROUND(1245.5556,1,0);
```

(No column name)

1245.6000

- Afkappen op 1 cijfer na de komma

```
SELECT ROUND(1245.5556,1,2);
```

(No column name)

1245.5000

Numerieke / wiskundige functies: afgerond, volgend - of vorig geheel getal bepalen

Voorbeelden:

```
SELECT
    bedrag,
    ROUND(bedrag,1) AS 'afgerond (0.1)',
    ROUND(bedrag,1,1) AS 'afgekapt (0.1)',
    FLOOR(bedrag) AS 'ondergrens',
    CEILING(bedrag) AS 'bovengrens',
    bedrag + 0.5 AS 'bedrag + 0.50',
    ROUND(bedrag + 0.50,2) AS 'afgerond + 0.50 (0.01)',
    ROUND(bedrag + 0.50,2,3) AS 'afgekapt + 0.50 (0.01)',
    FLOOR(bedrag + 0.50) AS 'ondergrens + 0.50',
    CEILING(bedrag + 0.50) AS 'bovengrens + 0.50'
FROM Tennis.boete;
```

Numerieke / wiskundige functies: afgerond, volgend - of vorig geheel getal bepalen

Voorbeelden:

bedrag	afgerond (0.1)	afgekapt (0.1)	ondergrens	bovengrens
100,1234	100,10	100,10	100,00	101,00
75,5678	75,60	75,50	75,00	76,00
100,789	100,80	100,70	100,00	101,00
50,2345	50,20	50,20	50,00	51,00
25,5487	25,50	25,50	25,00	26,00
25,0045	25,00	25,00	25,00	26,00
30,0078	30,00	30,00	30,00	31,00
75,0457	75,00	75,00	75,00	76,00

bedrag + 0.50	afgerond + 0.50 (0.01)	afgekapt + 0.50 (0.01)	ondergrens + 0.50	bovengrens + 0.50
100.6234	100.6200	100.6200	100	101
76.0678	76.0700	76.0600	76	77
101.2890	101.2900	101.2800	101	102
50.7345	50.7300	50.7300	50	51
26.0487	26.0500	26.0400	26	27
25.5045	25.5000	25.5000	25	26
30.5078	30.5100	30.5000	30	31
75.5457	75.5500	75.5400	75	76

Numerieke / wiskundige functies: willekeurige getal ophalen

Gebruik:

- Geeft een willekeurig decimaal getal tussen 0 (inclusief) en 1(exclusief)

Syntax:

- `RAND()`

willekeurig getal ≥ 0 en < 1	willekeurig getal ≥ 5 en < 6	willekeurig getal ≥ 5 en < 15
0,787295977157808	5,90561269689723	14,3031612804808

willekeurig getal ≥ 0 en < 1	willekeurig getal ≥ 5 en < 6	willekeurig getal ≥ 5 en < 15
0,280193226598423	5,03474587389909	7,96292607366486

Voorbeelden:

```
SELECT
  RAND() AS 'willekeurig getal  $\geq 0$  en  $< 1$ ',
  RAND() + 5 AS 'willekeurig getal  $\geq 5$  en  $< 6$ ',
  RAND() * 10 + 5 AS 'willekeurig getal  $\geq 5$  en  $< 15$ '
;
```

Datum en tijd functies

We bespreken de meest gebruikte:

- DAY()
- DATENAME()
- CURRENT_TIMESTAMP
- DATEFROMPARTS()
- DATEDIFF()
- DATEADD()
- ISDATE()

- MONTH()
- DATEPART()
- GETDATE()

- YEAR()
- GETUTCDATE()

Meer informatie over de andere functies:

https://www.w3schools.com/sql/sql_ref_sqlserver.asp

Datum en tijdfuncties: dag, maand of jaar ophalen uit een datum

Gebruik:

- Geef de dag, maand of jaar voor een bepaalde datum afhankelijk van de gebruikte functie

Syntax:

- `DAY(date)` → *getal van 1 t/m 31*
- `MONTH(date)` → *getal van 1 t/m 12*
- `YEAR(date)`

Datum en tijdfuncties: dag, maand of jaar ophalen uit een datum

Voorbeeld

```
SELECT
  datum,
  DAY(datum) AS 'dag',
  MONTH(datum) AS 'maand',
  YEAR(datum) AS 'jaar'
FROM Tennis.Boete;
```

datum	dag	maand	jaar
2000-12-08	8	12	2000
2001-05-05	5	5	2001
2003-09-10	10	9	2003
2004-12-08	8	12	2004
2000-12-08	8	12	2000
2000-12-08	8	12	2000
2002-12-30	30	12	2002
2004-11-12	12	11	2004

Datum en tijdfuncties: ophalen van een opgegeven deel

Gebruik:

- Geeft een opgegeven deel van een opgegeven datum als een **stringwaarde**
- Geeft een opgegeven deel van een opgegeven datum als een **integer**

Te gebruiken functies:

- DATENAME(*interval*, *datum*) → string
- DATEPART(*interval*, *datum*) → integer

Datum en tijdfuncties: ophalen van een opgegeven deel

Intervalwaarden:

- YEAR of YY of YYYY → jaar
- MONTH of MM of M → maand
- DAY of DY of Y → dag
- DAYOFYEAR → dag van het jaar
- WEEK of WW of WK → week
- WEEKDAY of DW of W → weekdag
- HOUR of HH → uur
- MINUTE of MI of N → minuut
- SECOND of SS of S → seconde
- MILLISECOND of MS → milliseconde

Datum en tijdfuncties: ophalen van een opgegeven deel

Voorbeeld:

```
SELECT
    naam,
    geboortedatum,
    DATENAME(WEEKDAY, geboortedatum) AS 'geboortedag string',
    DATEPART(WEEKDAY, geboortedatum) AS 'geboortedag integer'
FROM Tennis.Speler;
```

naam	geboortedatum	geboortedag string	geboortedag integer
Elfring	1988-09-01	Thursday	5
Permentier	1984-06-25	Monday	2
Wijers	1983-05-11	Wednesday	4
Niewenburg	1982-07-08	Thursday	5
Cools	1984-12-28	Friday	6
Cools	1983-06-22	Wednesday	4
Bischoff	1986-10-29	Wednesday	4
Bakker, de	1983-01-09	Sunday	1

Datum en tijdfuncties: huidige datum en tijd ophalen

Gebruik:

- Geef de **huidige datum en tijd**
in formaat 'JJJJ-MM-DD uu:mm:ss.mmm'

Te gebruiken functies:

- CURRENT_TIMESTAMP **!! Geen haken**
- GETDATE()
- GETUTCDATE()

Datum en tijdfuncties: huidige datum en tijd ophalen

Voorbeeld

```
SELECT  
    CURRENT_TIMESTAMP AS 'current timestamp',  
    GETDATE()          AS 'huidige lokale tijd',  
    GETUTCDATE()       AS 'huidige wereldtijd';
```

current timestamp	huidige lokale tijd	huidige wereldtijd
2018-11-29 10:29:38.020	2018-11-29 10:29:38.020	2018-11-29 09:29:38.020

Datum en tijdfuncties: datum samenstellen

Gebruik:

- Stelt een datum samen uit de opgegeven delen (jaar-, maand- en dagwaarden)

Syntax:

- **DATEFROMPARTS**(jaar, maand, dag)
 - jaar = 4 cijfers
 - maand = 1 t/m 12
 - dag = 1 t/m 31

Voorbeeld:

```
SELECT  
    DATEFROMPARTS('2019', '04', '21');
```

```
(No column name)  
2019-04-21
```

Datum en tijdfuncties: verschil tussen datums bepalen

Gebruik:

- Geef het **verschil** tussen twee datumwaarden

Syntax:

- **DATEDIFF**(*interval*, *datum1*, *datum2*)
- Interval:
 - YEAR = aantal jaren verschil
 - MONTH = aantal maanden verschil
 - DAY = aantal dagen verschil
 - ... https://www.w3schools.com/sql/func_sqlserver_datediff.asp

Datum en tijdfuncties: verschil tussen datums bepalen

Voorbeeld:

```
SELECT
    naam,
    geboortedatum,
    DATEDIFF(YEAR, geboortedatum, CURRENT_TIMESTAMP) AS 'leeftijd',
    CURRENT_TIMESTAMP AS 'Huidige datum tijd'
FROM Tennis.Speler;
```

naam	geboortedatum	leeftijd	Huidige datum tijd
Elfring	1988-09-01	30	2018-11-29 10:46:22.117
Permentier	1984-06-25	34	2018-11-29 10:46:22.117
Wijers	1983-05-11	35	2018-11-29 10:46:22.117
Niewenburg	1982-07-08	36	2018-11-29 10:46:22.117
Cools	1984-12-28	34	2018-11-29 10:46:22.117
Cools	1983-06-22	35	2018-11-29 10:46:22.117
Bischoff	1986-10-29	32	2018-11-29 10:46:22.117
Bakker, de	1983-01-09	35	2018-11-29 10:46:22.117

Datum en tijdfuncties: datums verhogen met een waarde

Gebruik:

- Geeft een datum nadat een bepaalde tijd / datuminterval is toegevoegd

Syntax:

- **DATEADD**(*interval*, *aantal*, *datum*)
- Interval:
 - YEAR = jaren optellen
 - MONTH = maanden optellen
 - DAY = dagen optellen
 - ... https://www.w3schools.com/sql/func_sqlserver_dateadd.asp

Datum en tijdfuncties: datums verhogen met een waarde

Voorbeeld:

```
SELECT  
    naam,  
    geboortedatum,  
    DATEADD(DAY,5,geboortedatum)  
FROM Tennis.Speler;
```

naam	geboortedatum	(No column name)
Elfring	1988-09-01	1988-09-06
Permentier	1984-06-25	1984-06-30
Wijers	1983-05-11	1983-05-16
Niewenburg	1982-07-08	1982-07-13
Cools	1984-12-28	1985-01-02

Datum en tijdfuncties: onderzoek of een waarde een datum is

Gebruik:

- Controleert of de uitdrukking een datum is
- Returnwaarde:
 - 1 = geldige datum
 - 0 = geen geldige datum

Syntax:

- ISDATE(expressie)

Datum en tijdfuncties: onderzoek of een waarde een datum is

Voorbeelden:

```
SELECT
  ISDATE(GETDATE())           AS 'huidige datum',
  ISDATE('huidige datum')    AS 'tekst ''huidige datum'',
  ISDATE('21/05/2016')       AS '21/05/2016',
  ISDATE('2016/05/18')       AS '2016/05/18',
  ISDATE('15 augustus 2019')  AS '15 augustus 2019',
  ISDATE('2019 augustus 15')  AS '2019 augustus 15',
  ISDATE('2019-08-15')       AS '2019-08-15';
```

huidige datum	tekst 'huidige datum'	21/05/2016	2016/05/18	15 augustus 2019	2019 augustus 15	2019-08-15
1	0	0	1	0	0	1

Geavanceerde functies

We bespreken:

- `CAST()`
- `CONVERT()`
- `ISNULL()`
- `ISNUMERIC()`
- `NULLIF()`

Geavanceerde functies: converteer naar een specifiek datatype

Gebruik:

- **Converteert** een uitdrukking van het ene **gegevenstype** naar een ander

Te gebruiken functies:

- CAST()
- CONVERT()

Geavanceerde functies: converteer naar een specifiek datatype

Syntax:

- **CAST**(*expressie AS data_type(lengte)*)
 - Expressie : verplicht
 - Datatype : verplicht
 - Lengte : optioneel
 - bij sommige datatype moet je ook een lengte opgeven bv. char, varchar, ...

Geavanceerde functies: converteer naar een specifiek datatype

Syntax:

- **CAST**(*expressie AS data_type(lengte)*)

Opmerking:

- Bij het converteren van een **float** of **numeriek** naar een geheel getal, zal de CAST () functie het resultaat **afkappen**.
- Voor **andere** conversies **rondt** de CAST () -functie het resultaat **af**.

Geavanceerde functies: converteer naar een specifiek datatype

Voorbeeld:

```
SELECT
  CAST(124.458 AS int),
  CAST('2018-12-20' AS date),
  CAST('Dit is een voorbeeld dat we een lengte moeten opgeven' AS
        varchar(25)),
  CAST('Dit is een voorbeeld dat we een lengte moeten opgeven' AS
        varchar),
  CAST('Dit is een voorbeeld dat we een lengte moeten opgeven' AS
        varchar(100));
```

Results		Messages			
	(No column name)	(No column name)	(No column name)	(No column name)	(No column name)
1	124	2018-12-20	Dit is een voorbeeld dat	Dit is een voorbeeld dat we ee	Dit is een voorbeeld dat we een lengte moeten opgeven

afgekapt

25

Default = 30

100

Geavanceerde functies: converteer naar een specifiek datatype

Syntax:

- **CONVERT**(*data_type(lengte)*, *expressie*, *style*)
 - Datatype : verplicht
 - Lengte : optioneel
 - bij sommige datatype moet je ook een lengte opgeven bv. char, varchar, ...
 - Expressie : verplicht
 - Style : optioneel
 - https://www.w3schools.com/sql/func_sqlserver_convert.asp

Geavanceerde functies: converteer naar een specifiek datatype

CONVERT() → Style

- Converteer van **datetime** naar **string (varchar)**

Zonder eeuw	Met eeuw	Input/output	standaard
0	100	mon dd yyyy hh:miAM/PM	standaard
3	103	dd/mm/yyyy	Brits/Frans
Andere waarden : zie https://www.w3schools.com/sql/func_sqlserver_convert.asp			

Geavanceerde functies: converteer naar een specifiek datatype

CONVERT() → Style

- Converteer van **datetime** naar **string (varchar)**

```
SELECT
    geboortedatum,
    CONVERT(varchar,geboortedatum,0),
    CONVERT(varchar,geboortedatum,3),
    CONVERT(varchar,geboortedatum,100),
    CONVERT(varchar,geboortedatum,103)
FROM Tennis.Speler;
```



geboortedatum	(No column name)	(No column name)	(No column name)	(No column name)
1988-09-01	Sep 1 1988	01/09/88	Sep 1 1988	01/09/1988
1984-06-25	Jun 25 1984	25/06/84	Jun 25 1984	25/06/1984
1983-05-11	May 11 1983	11/05/83	May 11 1983	11/05/1983
1982-07-08	Jul 8 1982	08/07/82	Jul 8 1982	08/07/1982
1984-12-28	Dec 28 1984	28/12/84	Dec 28 1984	28/12/1984

met eeuw

zonder eeuw

mon dd yyyy

dd/mm/yyyy

Geavanceerde functies: converteer naar een specifiek datatype

CONVERT() → Style

- Converteer van **money** naar **string (varchar)**

Waarde	Uitleg
0	Geen teken tussen de duizendtallen, 2 cijfers na het decimaalteken
1	Teken tussen de duizendtallen, 2 cijfers na het decimaalteken
2	Geen teken tussen de duizendtallen, 4 cijfers na het decimaalteken

Geavanceerde functies: converteer naar een specifiek datatype

CONVERT() → Style

- Converteer van **money** naar **string (varchar)**

```
SELECT
    salaris,
    CONVERT(varchar,salaris,0),
    CONVERT(varchar,salaris,1),
    CONVERT(varchar,salaris,2)
FROM dbo.Werknemer;
```

salaris	(No column name)	(No column name)	(No column name)
24000,00	24000.00	24,000.00	24000.0000
17000,00	17000.00	17,000.00	17000.0000
17000,00	17000.00	17,000.00	17000.0000
45000,00	45000.00	45,000.00	45000.0000
50000,00	50000.00	50,000.00	50000.0000

Geen teken tussen de
duizendtallen, 2 decimalen

Wel teken tussen de
duizendtallen, 2 decimalen

Geen teken tussen de duizendtallen, 4
decimalen

Geavanceerde functies: ISNULL()

Gebruik:

- Geeft een alternatieve waarde als de expressie NULL is.

Syntax:

- `ISNULL(expressie, alternatieveWaarde)`

Geavanceerde functies: ISNULL()

Voorbeelden

```
SELECT
    naam,
    voornaam,
    ISNULL(voornaam, 'Onbekende voornaam')
FROM DM_Oef5.Personeelslid;
```

 Results  Messages			
	naam	voornaam	(No column name)
1	De Coninck	Johan	Johan
2	Maes	Frederik	Frederik
3	Peeters	Jan	Jan
4	Claessen	NULL	Onbekende voornaam
5	Put	Jan	Jan

Geavanceerde functies: ISNULL()

Voorbeelden

```
SELECT
    naam,
    medewerkerId AS 'chef',
    ISNULL(CONVERT(varchar, medewerkerId), 'Onbekend')
FROM DM_Oef4.Afdeling;
```

	naam	chef	(No column name)
1	Analyse	1	1
2	Programmering	2	2
3	Inkoop	8	8
4	Productie Plant A	NULL	Onbekend
5	Productie Plant B	NULL	Onbekend
6	Verkoop	10	10

medewerkerId eerst
converteren van **int**
naar een **varchar**

Geavanceerde functies: ISNUMERIC()

Gebruik:

- **Controleert** of de uitdrukking een **geldig nummer** is.
- Geeft 1 als de uitdrukking een geldig nummer is, anders 0.

Syntax:

- ISNUMERIC(*expressie*)

Geavanceerde functies: ISNUMERIC()

Voorbeeld:

```
SELECT
    voornaam,
    ISNUMERIC(voornaam),
    personeelsnummer,
    ISNUMERIC(personeelsnummer)
FROM DM_Oef5.Personeelslid;
```

voornaam	(No column name)	personeelsnummer	(No column name)
Johan	0	1302	1
Frederik	0	1308	1
Jan	0	1304	1
NULL	0	1306	1
Jan	0	1317	1

Geavanceerde functies: NULLIF()

Gebruik:

- Vergelijkt twee uitdrukkingen.
- Als *expressie1* gelijk is aan *expressie2*
 - geeft de functie NULL terug,
 - anders *expressie1*

Syntax:

- `NULLIF(expressie1,expressie2)`

Geavanceerde functies: NULLIF()

Voorbeeld:

naam	lengte van de naam	plaats	lengte van de plaats	resultaat
Elfring	7	Den Haag	8	7
Permentier	10	Den Haag	8	10
Wijers	6	Den Haag	8	6
Niewenburg	10	Rijswijk	8	10
Cools	5	Zoetermeer	10	5
Cools	5	Leiden	6	5
Bischoff	8	Den Haag	8	NULL
Bakker, de	10	Rijswijk	8	10
Bohemen, van	12	Den Haag	8	12
Hofland	7	Den Haag	8	7
Meuleman	8	Voorburg	8	NULL
Permentier	10	Den Haag	8	10
Moerman	7	Zoetermeer	10	7
Baalen, van	11	Rotterdam	9	11
Van_Den_Berg	12	Amsterdam	9	12

```
SELECT
    naam,
    LEN(naam) AS 'lengte van de naam',
    plaats,
    LEN(plaats) AS 'lengte van de plaats',
    NULLIF(LEN(naam), LEN(plaats))
        AS 'resultaat'
FROM Tennis.Speler;
```

ORDER BY component

- Een sortering mag ook bestaan uit scalaire expressies
- De expressie wordt eerst uitgerekend voor elke rij, en dan gesorteerd

```
SELECT  
    naam,  
    voornaam,  
    id AS 'spelersnummer'  
FROM Tennis.Speler  
ORDER BY SUBSTRING(naam,1,1)
```

ORDER BY component

Originele volgorde

naam	voornaam	spelersnummer
Elfring	Rudi	2
Permentier	Raf	6
Wijers	Geert-Jan	7
Niewenburg	Bertha	8
Cools	Marjan	27
Cools	Carla	28
Bischoff	Danny	39
Bakker, de	Eric	44
Bohemen, van	Marc	57
Hofland	Pieter	83
Meuleman	Peter	95
Permentier	Patrick	100
Moerman	Daniëlla	104
Baalen, van	Ingrid	112
Van_Den_Berg	Lydia	113

Gevraagde volgorde

ORDER BY SUBSTRING(naam,1,1)

naam	voornaam	spelersnummer
Bischoff	Danny	39
Bakker, de	Eric	44
Bohemen, van	Marc	57
Baalen, van	Ingrid	112
Cools	Marjan	27
Cools	Carla	28
Elfring	Rudi	2
Hofland	Pieter	83
Meuleman	Peter	95
Moerman	Daniëlla	104
Niewenburg	Bertha	8
Permentier	Patrick	100
Permentier	Raf	6
Van_Den_Berg	Lydia	113
Wijers	Geert-Jan	7

Scalaire functies in de praktijk

Gebruikt in **SELECT**-statement.

- Het resultaat is **tijdelijk**.

Gebruikt in **UPDATE**-statement.

- Het resultaat is **definitief**

Scalaire functies in de praktijk

Voorbeeld: Originele inhoud tabel 'Voorwerp'

id	voorwerp	nieuwVoorwerp
1	potlood	potlood
2	tafel	tafel
3	stoel	stoel
4	lamp	lamp
5	bijzettafel	bijzettafel

Scalaire functies in de praktijk

Voorbeeld: SELECT + scalaire functie

```
SELECT  
    voorwerp,  
    nieuwVoorwerp,  
    UPPER(nieuwVoorwerp) AS 'Tijdelijke waarde'  
FROM dbo.Voorwerp;
```

voorwerp	nieuwVoorwerp	Tijdelijke waarde
potlood	potlood	POTLOOD
tafel	tafel	TAFEL
stoel	stoel	STOEL
lamp	lamp	LAMP
bijzettafel	bijzettafel	BIJZETTAFEL

Scalaire functies in de praktijk

Voorbeeld: UPDATE + scalaire functie

```
UPDATE dbo.Voorwerp  
    SET nieuwVoorwerp = UPPER(nieuwVoorwerp);
```

```
SELECT  
*  
FROM dbo.Voorwerp;
```

id	voorwerp	nieuwVoorwerp
1	potlood	POTLOOD
2	tafel	TAFEL
3	stoel	STOEL
4	lamp	LAMP
5	bijzettafel	BIJZETTAFEL